Brain Tumor Segmentation on MRI with a UNet

This is an individual project where I built a deep learning model that outlines brain tumors on MRI slices. Given one MRI slice, the model draws a mask over the pixels it thinks are tumor.

I did this as a beginner in machine learning. The project builds on what I learned during my internship at SGBC, where I worked with brain MRI, and on an NPTEL course. My goal was to learn the full machine learning workflow in medical imaging, from raw files to an honest evaluation.

What the model does

Input: one grayscale MRI slice, 512 by 512 pixels. Output: a mask of the same size, where each pixel is marked tumor or not tumor.

This is called segmentation. It answers "where is the tumor", not "what kind of tumor is it". Every slice in this dataset contains a tumor, so the model was never trained on healthy scans and cannot say whether a scan is healthy.

Dataset

I used the public brain tumor dataset from Jun Cheng on Figshare:
https://figshare.com/articles/dataset/brain_tumor_dataset/1512427

- 3,064 MRI slices from 233 patients, each with a tumor mask drawn by experts.
- Three tumor types: meningioma (708 slices), glioma (1,426 slices) and pituitary (930 slices).
- The slices are T1 weighted, contrast enhanced MRI according to the dataset description.
- Each patient has between 1 and 38 slices.
- 3,049 slices are 512 by 512 pixels. 15 slices are 256 by 256 and were enlarged to 512 by 512.
- The slices come from different viewing angles, not only the standard top down view. The model has to cope with that.

Please read the dataset page for its terms of use. The data files are not included in this repository.

Method

Preprocessing

- Each slice was scaled so its pixel values run from 0 to 1, using that slice's own minimum and maximum.
- The 15 smaller slices were resized to 512 by 512. Images used smooth (bilinear) resizing. Masks used nearest neighbour resizing so they stay strictly 0 or 1.
- Masks were converted to a simple tumor or not tumor label.

Train, validation and test split

The split is done by patient, not by slice. Slices from one patient look almost the same as each other. If they were shuffled randomly, some would land in training and their neighbours in testing, and the score would look better than it really is because the model has effectively seen the test patient already. This mistake is called data leakage.

- 163 patients for training (2,150 slices), 35 for validation (494 slices), 35 for testing (420 slices).

- The split keeps a similar mix of the three tumor types in every set.
- A check in the code confirms that no patient appears in more than one set. The overlap is 0 for every pair.
- The random seed is fixed so the same split is rebuilt every time.

Model

A UNet, a standard design for medical image segmentation, written from scratch in PyTorch.

- The first half repeatedly shrinks the image (512, 256, 128, 64, 32 pixels wide) while learning more abstract features (16, 32, 64, 128, 256 channels).
- The second half enlarges the image back to 512 by 512 to mark every pixel.
- Skip connections copy the sharp, detailed early features straight across to the matching enlarging step. This keeps the tumor edges from becoming blurry.
- The last layer gives one score per pixel. A sigmoid turns it into a probability, and probabilities above 0.5 become tumor.

Loss and metrics

- Loss: Dice loss plus binary cross entropy. Dice loss rewards overlap between the predicted and true tumor. Binary cross entropy checks each pixel and helps early training.
- Metrics: Dice and IoU (intersection over union). Both measure overlap between the predicted and true mask, from 0 (no overlap) to 1 (perfect). Pixel accuracy is not used, because the tumor is a small part of each image and a model that predicts nothing would still look accurate.

Training

- Optimizer: Adam, learning rate 0.001, batch size 4, 20 epochs.
- Augmentation: random horizontal flips, applied to the image and mask together.
- Mixed precision on a GPU to save memory and time. Trained on a free Google Colab T4 GPU.
- The weights from the epoch with the best validation Dice were kept (epoch 16, validation Dice 0.664). The test set was not used for any choices.

Results

Scores on the 420 test slices from 35 patients the model never saw:

- Overall: Dice 0.643, IoU 0.539
- Meningioma (112 slices): Dice 0.840, IoU 0.755
- Pituitary (120 slices): Dice 0.705, IoU 0.601
- Glioma (188 slices): Dice 0.486, IoU 0.371

Sample predictions are saved by `evaluate.py` as `results/overlay.png`. In the examples I looked at, the predicted shape usually sat on the true tumor with a similar size. One failure case showed the model finding the real tumor but also marking a second, wrong region, which lowers the score for that slice.

Glioma was the hardest type. My guess is that gliomas have irregular edges that blend into the surrounding brain, while meningiomas are compact and clearly bounded. I have not tested this explanation.

Limitations

- This is a baseline. It uses a small UNet trained for only 20 epochs with no tuning, so the scores can likely be improved.
- The test set is only 35 patients and I used a single split, so the scores would shift somewhat with a different split.
- The model works on single 2D slices. It does not use the 3D structure of a scan.
- The model only uses one MRI type (T1 weighted, contrast enhanced).
- The mixed viewing angles and the 15 enlarged slices were not handled specially.
- The reported numbers come from one run without a fixed training seed. The scripts now set a seed, so a rerun will give similar but not identical numbers.
- This is a learning project and has not been validated for any clinical use.

Planned next steps

- Train for longer, with a learning rate schedule and more augmentation, and judge changes on validation Dice only.
- Compare 256 by 256 inputs with 512 by 512.
- Extend the project into a full three part pipeline. All of this is planned and not yet done:
  - Detection: add healthy MRI scans and train the model to say whether a scan contains a tumor or not. This dataset has no healthy scans, so healthy scans would have to come from another public source, and I would check that differences between scanners and scan settings are not what the model learns instead of the tumor.
  - Localization: keep the current segmentation model to mark the tumor region on scans where a tumor is found.
  - Classification: predict the tumor type (meningioma, glioma or pituitary), using the same patient split.

How to run

1. Install the requirements: `pip install -r requirements.txt`
2. Download the four zip files from the dataset page and unzip them into one folder.
3. Build the arrays and the split: `python preprocess.py --raw path/to/mat/files --out data`
4. Train: `python train.py --data data --epochs 20 --out best.pt`
5. Evaluate once on the test patients: `python evaluate.py --data data --weights best.pt`

A GPU is strongly recommended. At 512 by 512, one training epoch took roughly a minute on a Colab T4.

Files

- `preprocess.py` reads the raw files, resizes and scales them, and makes the patient split.
- `common.py` holds the dataset class, the UNet, the loss and the metrics.
- `train.py` trains the model and saves the best weights.
- `evaluate.py` scores the model on the test patients and saves example predictions.